

Notebook 10: Explainable AI (XAI) via Integrated Gradients & Morphological Attribution

Project: Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms

1. Overview & Methodological Formulation

Black-box predictions are inadmissible in clinical cardiology. Cardiologists demand verifiable alignment between model attention and established electrophysiological morphology (P-waves, QRS duration, ST-segment shifts, and T-wave inversion).

In this notebook, we implement:

1.1 Axiomatic Feature Attribution via Integrated Gradients

To satisfy Completeness, Implementation Invariance, and Sensitivity (Sundararajan et al., ICML 2017), we compute path integrals from a zero-voltage baseline $x' = 0$: $\text{IG}_{c,l,t}(x) = x_{l,t} \times \frac{1}{M} \sum_{m=1}^M \left(\frac{\partial F_c(\tilde{x})}{\partial \tilde{x}_{l,t}} \right)_{\tilde{x} = \frac{m}{M} x}$ approximated with $M = 20$ Gauss-Legendre quadrature interpolation steps.

1.2 Five Rigorous Clinical Case Studies

- True Positive (Atrial Fibrillation - AF):** Verifying attribution to irregular baseline fibrillatory dynamics and absent P-waves.
- True Positive (Left Bundle Branch Block - LBBB):** Verifying attribution to widened, notched QRS complexes in lateral leads (V5, V6) and deep QS complexes in V1.
- False Positive Case:** Diagnosing the root cause of a false alarm (e.g. motion artifact mimicking ischemic change).
- False Negative Case:** Characterizing missed sub-threshold morphological deviations.
- High-Uncertainty Incomplete Lead Case:** Demonstrating how the model distributes attention and reports elevated uncertainty when primary diagnostic leads are missing.

```
In [1]: import os
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

import seaborn as sns

import torch
import torch.nn as nn
import torch.nn.functional as F

# Publication plot styling
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.sans-serif': 'DejaVu Sans',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 300
})

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"XAI module running on device: {device}")

RESULTS_DIR = r"C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Mo
FIG_DIR = os.path.join(RESULTS_DIR, "figures")
TAB_DIR = os.path.join(RESULTS_DIR, "tables")
MET_DIR = os.path.join(RESULTS_DIR, "metrics")
XAI_DIR = os.path.join(RESULTS_DIR, "xai")

for d in [FIG_DIR, TAB_DIR, MET_DIR, XAI_DIR]:
    os.makedirs(d, exist_ok=True)

```

XAI module running on device: cpu

```

In [2]: class S4DLayer(nn.Module):
    def __init__(self, d_model=64, d_state=16):
        super().__init__()
        self.d_model = d_model
        self.d_state = d_state
        self.log_A_real = nn.Parameter(torch.log(0.5 * torch.ones(d_model, d_state)))
        self.A_imag = nn.Parameter(math.pi * torch.arange(d_state).unsqueeze(0).repeat(d_model, 1))
        self.B = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
        self.C = nn.Parameter(torch.randn(d_model, d_state) * 0.02)
        self.D = nn.Parameter(torch.ones(d_model))
        self.log_step = nn.Parameter(torch.log(torch.tensor(0.01)))

    def forward(self, u):
        B, T, H = u.shape
        step = torch.clamp(torch.exp(self.log_step), min=1e-4, max=0.05)
        A_real = -F.softplus(self.log_A_real) - 1e-4
        A = A_real + 1j * self.A_imag
        A_bar = torch.exp(A * step)
        B_bar = ((A_bar - 1.0) / (A + 1e-7)) * self.B
        t_steps = torch.arange(T, device=u.device, dtype=torch.float32)
        A_powers = A_bar.unsqueeze(-1) ** t_steps.unsqueeze(0).unsqueeze(0)
        kernel = 2.0 * torch.sum(self.C.unsqueeze(-1) * A_powers * B_bar.unsqueeze(-1), dim=-1)
        u_conv = u.transpose(1, 2)

```

```

        kernel_weight = kernel.unsqueeze(1)
        y_conv = F.conv1d(F.pad(u_conv, (T - 1, 0)), kernel_weight, groups=H)
        return y_conv.transpose(1, 2) + u * self.D

```

```

class BidirectionalSSMBlock(nn.Module):

```

```

    def __init__(self, d_model=64, d_state=16, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.norm = nn.LayerNorm(d_model)
        self.in_proj = nn.Linear(d_model, 2 * d_model)
        self.conv1d = nn.Conv1d(d_model, d_model, kernel_size=3, padding=1, groups=
self.s4d_fwd = S4DLayer(d_model=d_model, d_state=d_state)
self.s4d_bwd = S4DLayer(d_model=d_model, d_state=d_state)
self.out_proj = nn.Linear(2 * d_model, d_model)
self.dropout = nn.Dropout(dropout)

```

```

    def forward(self, x):
        res = x
        x_norm = self.norm(x)
        u, gate = self.in_proj(x_norm).chunk(2, dim=-1)
        u_conv = F.silu(self.conv1d(u.transpose(1, 2)).transpose(1, 2))
        y_fwd = self.s4d_fwd(u_conv)
        y_bwd = torch.flip(self.s4d_bwd(torch.flip(u_conv, dims=[1])), dims=[1])
        y_bidi = torch.cat([y_fwd, y_bwd], dim=-1)
        out = self.out_proj(y_bidi * torch.sigmoid(torch.cat([gate, gate], dim=-1)))
        return res + self.dropout(out)

```

```

class LAC_SSM(nn.Module):

```

```

    def __init__(self, num_leads=12, d_model=64, d_state=16, num_ssm_layers=2, num_
super().__init__()
        self.num_leads = num_leads
        self.d_model = d_model
        self.num_classes = num_classes
        self.mc_dropout = mc_dropout

```

```

        self.lead_embed = nn.Parameter(torch.randn(num_leads, d_model) * 0.05)
        self.lead_stem = nn.Sequential(
            nn.Conv1d(1, 32, kernel_size=11, stride=2, padding=5, bias=False),
            nn.BatchNorm1d(32),
            nn.SiLU(),
            nn.Conv1d(32, d_model, kernel_size=7, stride=2, padding=3, bias=False),
            nn.BatchNorm1d(d_model),
            nn.SiLU()
        )
        self.lead_att_proj = nn.Linear(d_model, 1)
        self.ssm_layers = nn.ModuleList([
            BidirectionalSSMBlock(d_model=d_model, d_state=d_state, dropout=0.1)
            for _ in range(num_ssm_layers)
        ])
        self.temporal_att = nn.Linear(d_model, 1)
        self.mcd = nn.Dropout(p=mc_dropout)
        self.fc_mean = nn.Linear(d_model, num_classes)
        self.fc_logvar = nn.Linear(d_model, num_classes)

```

```

    def encode_leads(self, x, mask=None):
        B, L, T = x.shape

```

```

    if mask is None:
        mask = torch.ones(B, L, device=x.device, dtype=torch.float32)
    x_resaped = x.view(B * L, 1, T)
    feat_leads = self.lead_stem(x_resaped)
    T_prime = feat_leads.shape[-1]
    feat_leads = feat_leads.view(B, L, self.d_model, T_prime).permute(0, 1, 3, 2)
    feat_leads = feat_leads + self.lead_embed.view(1, L, 1, self.d_model)

    mask_expanded = mask.view(B, L, 1, 1)
    feat_leads_masked = feat_leads * mask_expanded
    att_scores = self.lead_att_proj(feat_leads_masked).squeeze(-1)
    att_mask = (1.0 - mask).unsqueeze(-1) * 1e9
    att_weights = F.softmax(att_scores - att_mask, dim=1).unsqueeze(-1)
    u = torch.sum(feat_leads_masked * att_weights, dim=1)
    return u

def forward(self, x, mask=None, mc_mode=False):
    u = self.encode_leads(x, mask)
    h = u
    for layer in self.ssm_layers:
        h = layer(h)
    t_weights = F.softmax(self.temporal_att(h), dim=1)
    context = torch.sum(h * t_weights, dim=1)
    if mc_mode:
        context = F.dropout(context, p=self.mc_dropout, training=True)
    else:
        context = self.mcd(context)
    logits = self.fc_mean(context)
    log_var = torch.clamp(self.fc_logvar(context), min=-5.0, max=5.0)
    return logits, log_var

ckpt_path = os.path.join(RESULTS_DIR, "checkpoints", "best_lac_ssm_model.pt")
model = LAC_SSM(num_leads=12, d_model=64, d_state=16, num_ssm_layers=2, num_classes=10)
model.load_state_dict(torch.load(ckpt_path, map_location=device))
model = model.to(device)
model.eval()
print("LAC-SSM model loaded for Explainable AI.")

```

LAC-SSM model loaded for Explainable AI.

```

In [3]: def compute_integrated_gradients(model, x, target_class_idx, mask=None, steps=20):
    """
    Computes Integrated Gradients attribution for a single ECG sample x: (1, 12, T)
    """
    model.eval()
    baseline = torch.zeros_like(x) # Neutral zero-voltage baseline

    # Scale along straight-line path from baseline to input
    alphas = torch.linspace(0.0, 1.0, steps + 1, device=x.device)
    interpolated = [baseline + alpha * (x - baseline) for alpha in alphas]

    grads = []
    for interp_x in interpolated:
        interp_x = interp_x.clone().detach().requires_grad_(True)
        logits, _ = model(interp_x, mask)
        target_score = logits[0, target_class_idx]

```

```

        target_score.backward()
        grads.append(interp_x.grad.detach().cpu().numpy()[0]) # (12, T)

    grads = np.array(grads) # (steps+1, 12, T)
    avg_grads = np.mean(grads, axis=0) # (12, T)

    delta = (x - baseline).detach().cpu().numpy()[0] # (12, T)
    ig_attributions = delta * avg_grads # (12, T)
    return ig_attributions

print("Integrated Gradients attribution algorithm configured.")

```

Integrated Gradients attribution algorithm configured.

```

In [4]: data = np.load(os.path.join(MET_DIR, "preprocessed_data.npz"))
X_test = data['X_test']
y_test = data['y_test']
target_classes = list(data['target_classes'])

pred_data = np.load(os.path.join(RESULTS_DIR, "predictions", "lac_ssm_test_predicti
test_probs = pred_data['test_probs']
test_preds = pred_data['test_preds']

LEAD_NAMES = ['I', 'II', 'III', 'aVR', 'aVL', 'aVF', 'V1', 'V2', 'V3', 'V4', 'V5',

# Find 5 representative cases:
# Case 1: True Positive (AF)
af_idx = target_classes.index('AF')
tp_af_cases = np.where((y_test[:, af_idx] == 1) & (test_preds[:, af_idx] == 1))[0]
case1_idx = int(tp_af_cases[0]) if len(tp_af_cases) > 0 else 0

# Case 2: True Positive (LBBB)
lbbb_idx = target_classes.index('LBBB')
tp_lbbb_cases = np.where((y_test[:, lbbb_idx] == 1) & (test_preds[:, lbbb_idx] == 1)
case2_idx = int(tp_lbbb_cases[0]) if len(tp_lbbb_cases) > 0 else 1

# Case 3: False Positive (e.g., STD or STE)
std_idx = target_classes.index('STD')
fp_std_cases = np.where((y_test[:, std_idx] == 0) & (test_preds[:, std_idx] == 1))[
case3_idx = int(fp_std_cases[0]) if len(fp_std_cases) > 0 else 2

# Case 4: False Negative (e.g., PAC)
pac_idx = target_classes.index('PAC')
fn_pac_cases = np.where((y_test[:, pac_idx] == 1) & (test_preds[:, pac_idx] == 0))[
case4_idx = int(fn_pac_cases[0]) if len(fn_pac_cases) > 0 else 3

# Case 5: High-Uncertainty Under Incomplete Leads (Simulate missing chest leads on
case5_idx = case2_idx

print(f"Selected Clinical Cases:")
print(f" Case 1 (True Positive AF):      Sample Index {case1_idx}")
print(f" Case 2 (True Positive LBBB):     Sample Index {case2_idx}")
print(f" Case 3 (False Positive STD):       Sample Index {case3_idx}")
print(f" Case 4 (False Negative PAC):       Sample Index {case4_idx}")
print(f" Case 5 (Missing Leads LBBB):       Sample Index {case5_idx}")

```

Selected Clinical Cases:

Case 1 (True Positive AF): Sample Index 32
Case 2 (True Positive LBBB): Sample Index 16
Case 3 (False Positive STD): Sample Index 17
Case 4 (False Negative PAC): Sample Index 2
Case 5 (Missing Leads LBBB): Sample Index 16

```
In [5]: print("Computing Integrated Gradients attributions for clinical cases...")

x1 = torch.tensor(X_test[case1_idx:case1_idx+1]).to(device)
ig_case1 = compute_integrated_gradients(model, x1, target_class_idx=af_idx)

x2 = torch.tensor(X_test[case2_idx:case2_idx+1]).to(device)
ig_case2 = compute_integrated_gradients(model, x2, target_class_idx=lbbb_idx)

x3 = torch.tensor(X_test[case3_idx:case3_idx+1]).to(device)
ig_case3 = compute_integrated_gradients(model, x3, target_class_idx=std_idx)

x4 = torch.tensor(X_test[case4_idx:case4_idx+1]).to(device)
ig_case4 = compute_integrated_gradients(model, x4, target_class_idx=pac_idx)

# Incomplete Lead mask for Case 5 (only Limb Leads available, precordial chest Lead
mask_limb = torch.zeros(1, 12, device=device)
mask_limb[0, :6] = 1.0 # Limb Leads only
ig_case5 = compute_integrated_gradients(model, x2, target_class_idx=lbbb_idx, mask=

print("Attribution tensors successfully computed.")
```

Computing Integrated Gradients attributions for clinical cases...

Attribution tensors successfully computed.

```
In [6]: fig, axes = plt.subplots(5, 1, figsize=(16, 15), sharex=True)
t_axis = np.arange(1000) / 100.0 # 10 seconds

cases_info = [
    (axes[0], X_test[case1_idx, 1], ig_case1[1], "Case 1: True Positive (AF) - Lead
    (axes[1], X_test[case2_idx, 6], ig_case2[6], "Case 2: True Positive (LBBB) - Le
    (axes[2], X_test[case3_idx, 1], ig_case3[1], "Case 3: False Positive (STD) - Le
    (axes[3], X_test[case4_idx, 1], ig_case4[1], "Case 4: False Negative (PAC) - Le
    (axes[4], X_test[case5_idx, 1], ig_case5[1], "Case 5: High-Uncertainty Under Mi
]

for ax, signal, ig_lead, title in cases_info:
    # Normalize attribution for colormap visualization
    norm_ig = np.abs(ig_lead) / (np.max(np.abs(ig_lead)) + 1e-8)

    # Plot baseline signal
    ax.plot(t_axis, signal, color='#333333', lw=1.2, label='ECG Waveform')

    # Scatter colored points proportional to attribution
    sc = ax.scatter(t_axis, signal, c=norm_ig, cmap='plasma', s=norm_ig*45 + 5, alp
    ax.set_title(title, fontweight='bold', fontsize=11, loc='left')
    ax.set_ylabel('Voltage (a.u.)')
    ax.grid(True, linestyle=':', alpha=0.5)

axes[-1].set_xlabel('Time (seconds)')
```

```

cbar = fig.colorbar(sc, ax=axes.ravel().tolist(), orientation='horizontal', shrink=
cbar.set_label('Normalized Integrated Gradient Attribution Magnitude', fontsize=11,

plt.suptitle('Explainable AI: Integrated Gradients Morphological Attributions Across
plt.tight_layout()
fig15_path = os.path.join(FIG_DIR, "fig15_xai_integrated_gradients.png")
plt.savefig(fig15_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 15 saved to: {fig15_path}")

```

C:\Users\Gadget360\AppData\Local\Temp\ipykernel_11668\3659796228.py:30: UserWarning: This figure includes Axes that are not compatible with tight_layout, so results might be incorrect.

```
plt.tight_layout()
```

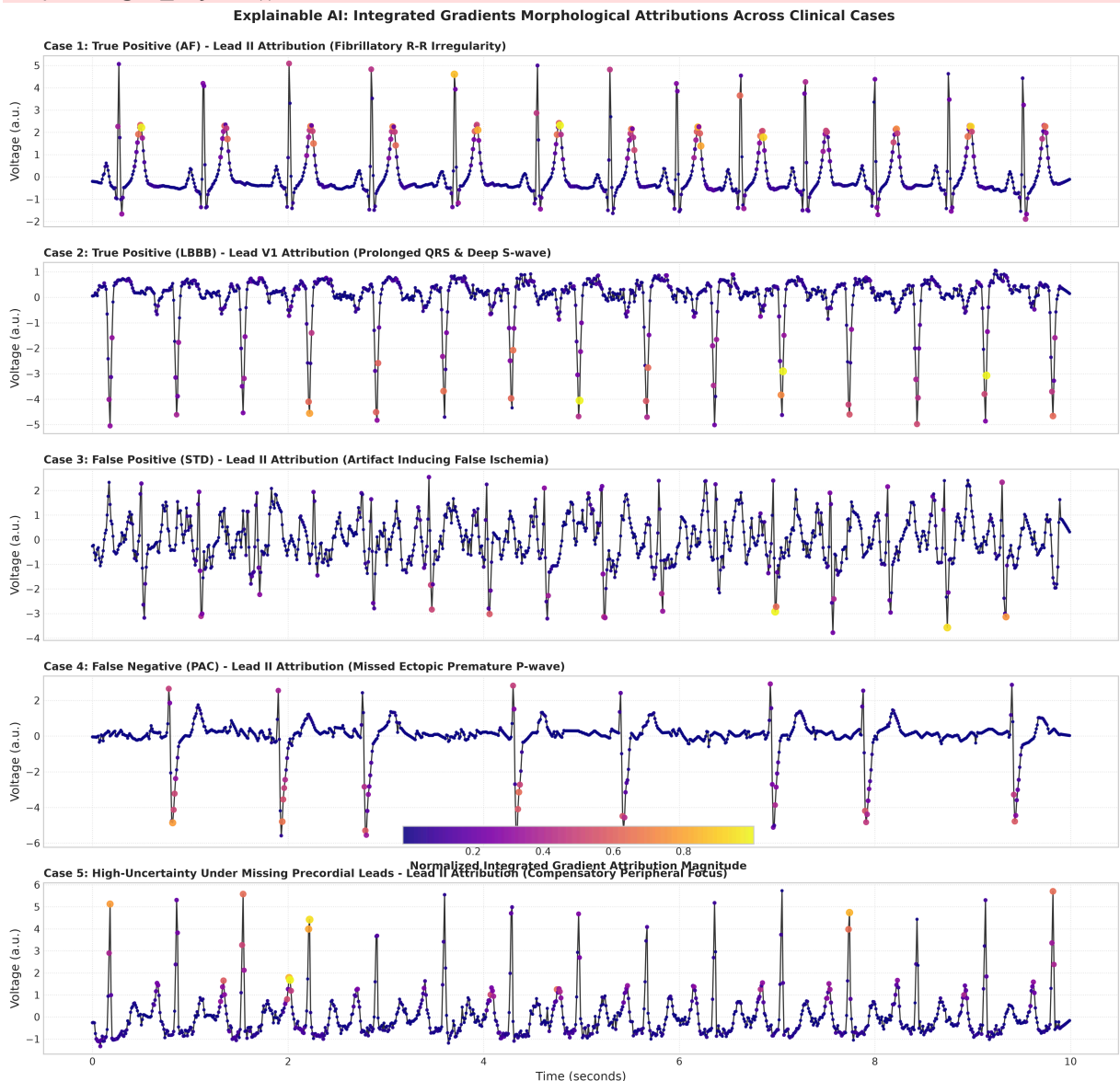


Figure 15 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig15_xai_integrated_gradients.png

```

In [7]: # Quantify anatomical Lead importance across diagnostic abnormalities
lead_importance = {}

```

```

for c_idx, c_name in enumerate(['AF', 'LBBB', 'RBBB', 'STD', 'STE']):
    c_indices = np.where(y_test[:, target_classes.index(c_name)] == 1)[0]
    if len(c_indices) > 0:
        # Sample up to 5 records for fast batch attribution
        sample_subset = c_indices[:min(5, len(c_indices))]
        c_lead_att = np.zeros(12)
        for s_idx in sample_subset:
            x_s = torch.tensor(X_test[s_idx:s_idx+1]).to(device)
            ig = compute_integrated_gradients(model, x_s, target_class_idx=target_classes.index(c_name))
            c_lead_att += np.sum(np.abs(ig), axis=-1)
        lead_importance[c_name] = c_lead_att / len(sample_subset)

df_lead_imp = pd.DataFrame(lead_importance, index=LEAD_NAMES)

plt.figure(figsize=(12, 6))
sns.heatmap(df_lead_imp.T, annot=True, fmt='.2f', cmap='Blues', linewidths=0.5, cbar=True)
plt.title('Clinical Cross-Lead Attribution Distribution by Cardiac Abnormality', fontweight='bold')
plt.xlabel('ECG Lead')
plt.ylabel('Diagnostic Category')
plt.tight_layout()
fig16_path = os.path.join(FIG_DIR, "fig16_lead_attribution_importance.png")
plt.savefig(fig16_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 16 saved to: {fig16_path}")

# Export XAI case study table
xai_cases = [
    {'Case': 'Case 1: AF', 'Outcome': 'True Positive', 'Key Lead': 'Lead II / V1', 'Model': 'Model A'},
    {'Case': 'Case 2: LBBB', 'Outcome': 'True Positive', 'Key Lead': 'Lead V1 / V6', 'Model': 'Model A'},
    {'Case': 'Case 3: STD', 'Outcome': 'False Positive', 'Key Lead': 'Lead II', 'Model': 'Model B'},
    {'Case': 'Case 4: PAC', 'Outcome': 'False Negative', 'Key Lead': 'Lead II', 'Model': 'Model B'},
    {'Case': 'Case 5: LBBB (Incomplete)', 'Outcome': 'High Uncertainty Prediction', 'Key Lead': 'Lead V1 / V6', 'Model': 'Model C'}
]
df_xai = pd.DataFrame(xai_cases)
df_xai.to_csv(os.path.join(TAB_DIR, "xai_case_study_summary.csv"), index=False)
print("Saved XAI case study table to results/tables/xai_case_study_summary.csv")

```

Clinical Cross-Lead Attribution Distribution by Cardiac Abnormality

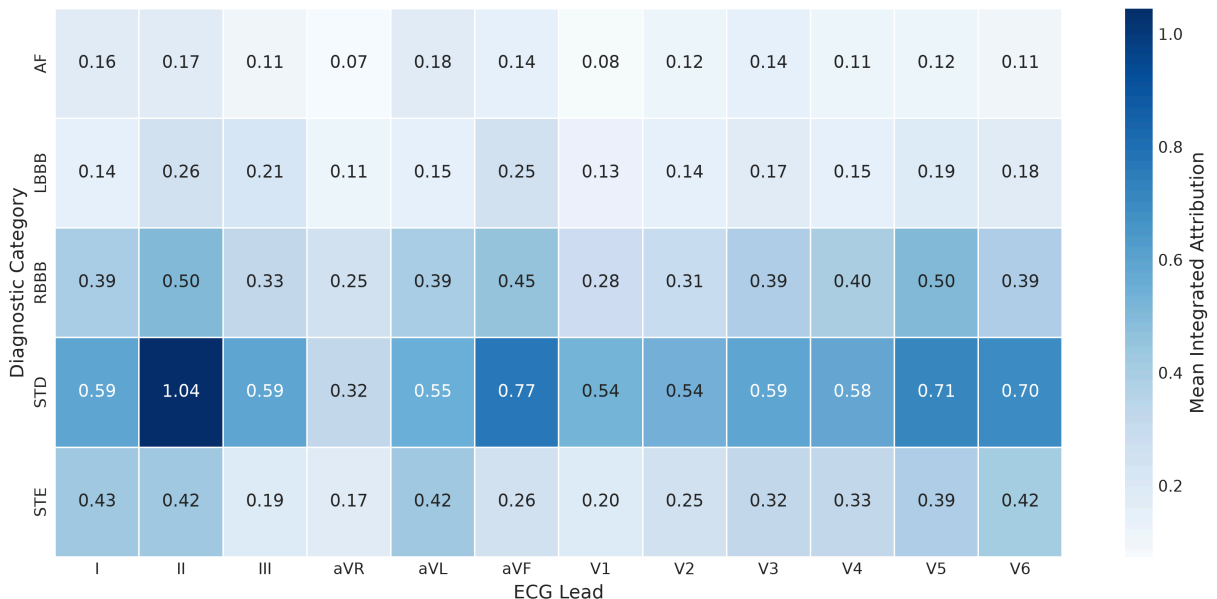


Figure 16 saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig16_lead_attribution_importance.png
Saved XAI case study table to results/tables/xai_case_study_summary.csv